

Optimization: Gradient Descent

Numerical Methods for Training AI Models

Firuz Kamalov

MAI 604: Deep Learning

February 26, 2026

- 1 Methods of Optimization
- 2 Analytic vs. Numeric Visualization
- 3 1D Example: A Simple Parabola
- 4 2D Example: A Paraboloid
- 5 Application: Linear Regression
- 6 Stochastic Gradient Descent (SGD)

1. Analytic Optimization

- Uses calculus to find exact solutions.
- Involves taking the derivative (gradient) of the function and setting it to zero.
- Solves the resulting equations to find max/min.
- **Advantage:** Provides an exact, closed-form solution.
- **Disadvantage:** Only feasible for simpler, well-behaved functions.

1. Analytic Optimization

- Uses calculus to find exact solutions.
- Involves taking the derivative (gradient) of the function and setting it to zero.
- Solves the resulting equations to find max/min.
- **Advantage:** Provides an exact, closed-form solution.
- **Disadvantage:** Only feasible for simpler, well-behaved functions.

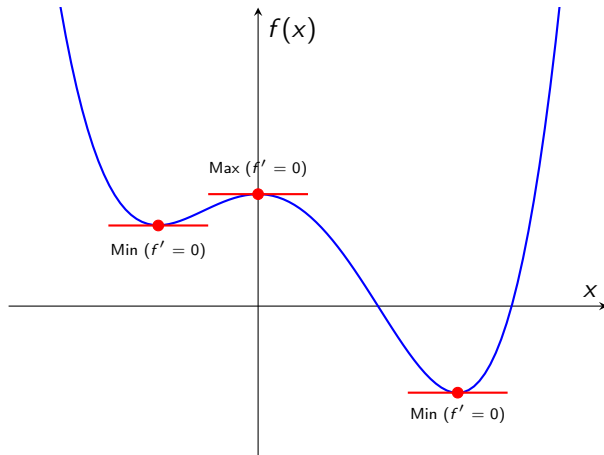
2. Numeric Optimization

- Uses iterative algorithms to find approximate solutions.
- Starts with an initial guess and repeatedly refines it to get closer to a minimum.
- Gradient Descent is a key example.
- **Advantage:** Can be applied to very complex functions where analytic solutions are intractable.
- **Disadvantage:** Solution is an approximation; performance depends on algorithm parameters.

1. Analytic Approach: Finding Critical Points

The Strategy:

- We look for points where the **slope (derivative) is zero**.



In general, we look for points where the **gradient is zero**:

$$\nabla f = \mathbf{0}$$

[Interactive 3D](#)

2. Numeric Approach: Gradient Descent Idea

The Intuition:

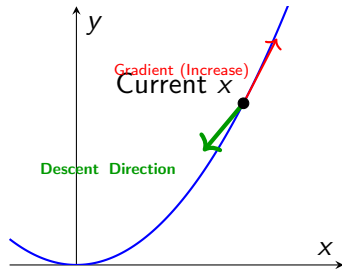
- The derivative $f'(x)$ gives the slope of the graph.
- The sign of the slope tells us the direction of **increase**.

The Algorithm: To find the *minimum*, we must move in the **opposite** direction of the derivative.

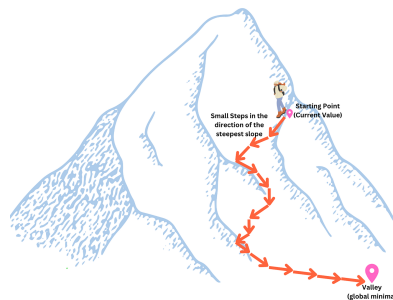
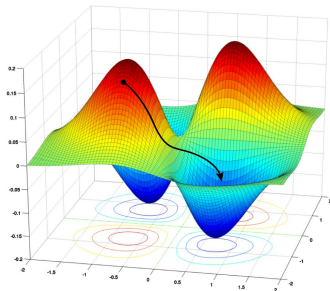
Update Rule

$$x_{new} = x_{old} - \eta \cdot f'(x_{old})$$

Where α is the learning rate (step size).



Gradient descent visualization



Update Rule (vector form):

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} - \eta \nabla f(\mathbf{x}_{\text{old}})$$

Gradient Descent Flow

1D Example - Analytic Solution

Consider the function $f(x) = x^2 - 4x$.

Solving Analytically:

- 1 Find the derivative of $f(x)$:

$$f'(x) = 2x - 4$$

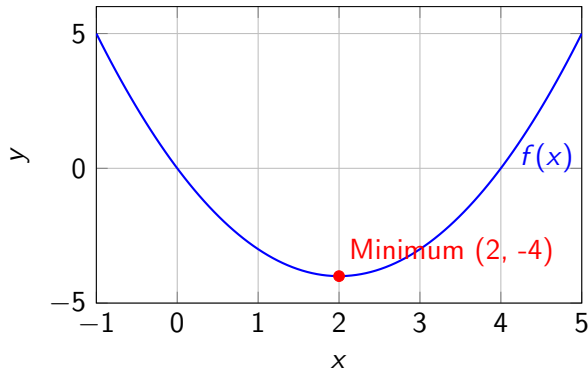
- 2 Set the derivative to zero to find critical points:

$$2x - 4 = 0$$

- 3 Solve for x :

$$2x = 4 \implies x = 2$$

The minimum is at $x = 2$, where
 $f(2) = 2^2 - 4(2) = -4$.



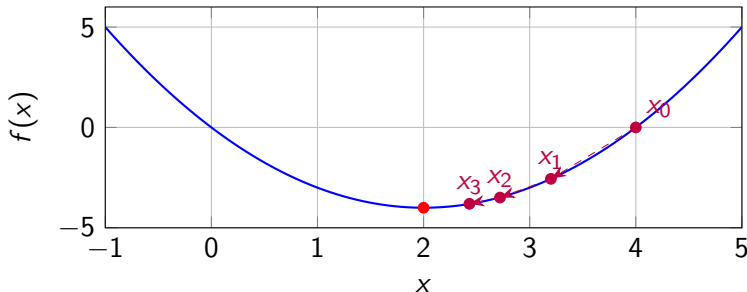
1D Example - Numeric (Gradient Descent)

Now, let's solve $f(x) = x^2 - 4x$ numerically.

- **Update Rule:** $x_{i+1} = x_i - \eta f'(x_i)$, where $f'(x) = 2x - 4$.
- **Parameters:** Let's start at a random point $x_0 = 4$ with a learning rate $\eta = 0.2$.

Iterations:

- $i = 0$: $x_1 = x_0 - 0.2 \cdot (2x_0 - 4) = 3.2$
- $i = 1$: $x_2 = x_1 - 0.2 \cdot (2x_1 - 4) = 2.72$
- $i = 2$: $x_3 = x_2 - 0.2 \cdot (2x_2 - 4) = 2.432$

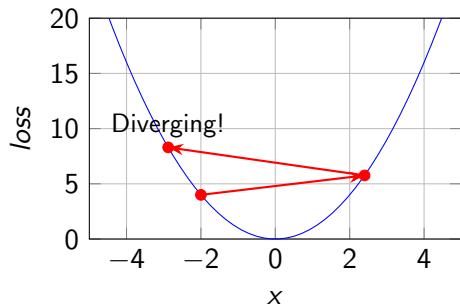


Importance of Learning Rate (η)

The learning rate η determines the step size at each iteration.

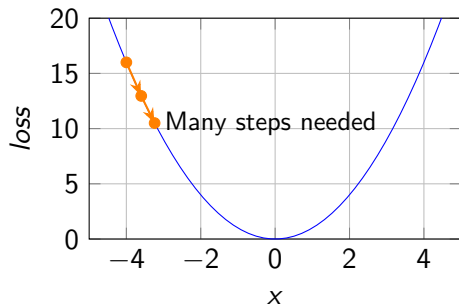
a) Large η : Overshooting

$\eta = 1.1$ (Divergence/Oscillation)



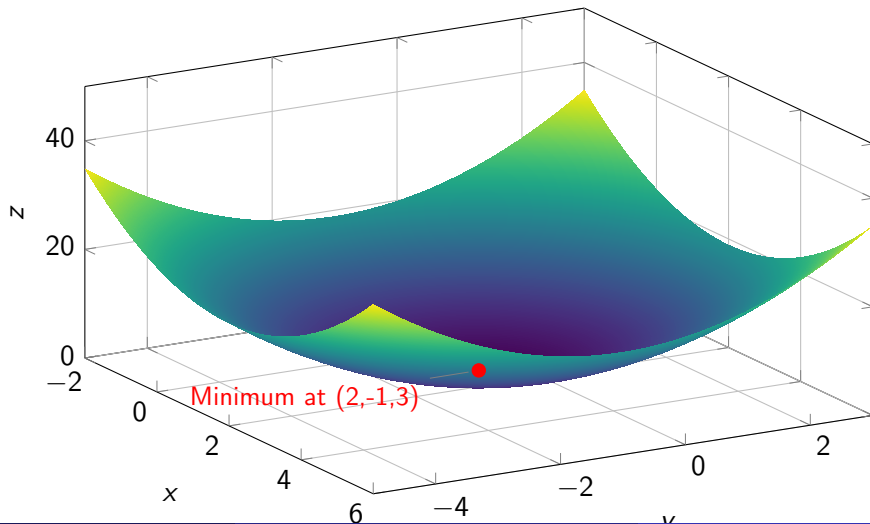
b) Small η : Slow Convergence

$\eta = 0.05$ (Crawling)



2D Example: $f(x, y) = x^2 + y^2 - 4x + 2y + 8$

A Convex Paraboloid



2D Example - Analytic Solution

For $f(x, y) = x^2 + y^2 - 4x + 2y + 8$.

Solving Analytically:

- 1 Find the gradient, $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$:

$$\frac{\partial f}{\partial x} = 2x - 4$$

$$\frac{\partial f}{\partial y} = 2y + 2$$

So, $\nabla f(x, y) = [2x - 4, 2y + 2]$.

- 2 Set the gradient to the zero vector, $\nabla f = [0, 0]$:

$$2x - 4 = 0 \implies x = 2$$

$$2y + 2 = 0 \implies y = -1$$

The minimum is at the point $(2, -1)$, where $f(2, -1) = 3$.

2D Example - Numeric (Gradient Descent)

For $f(x, y) = x^2 + y^2 - 4x + 2y + 8$.

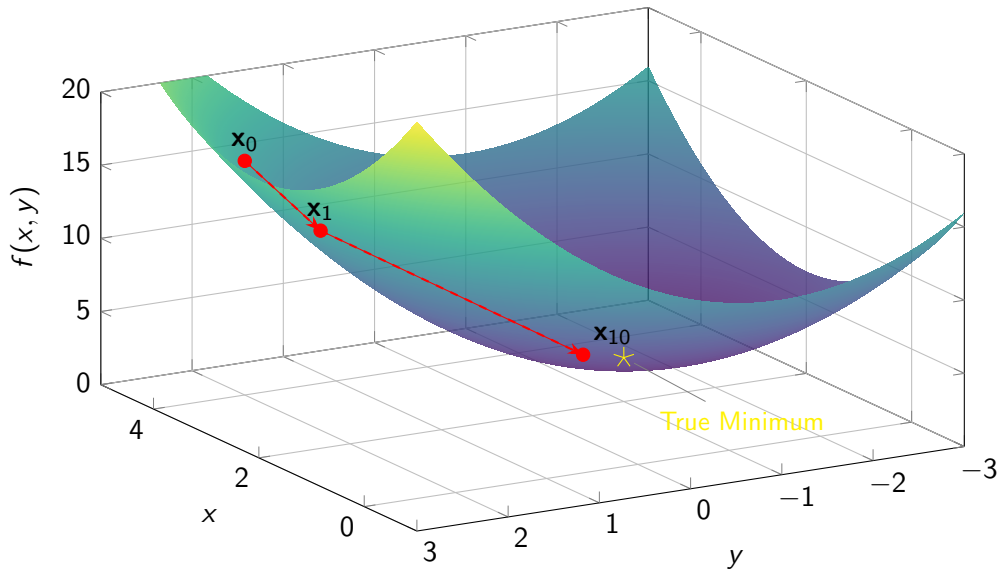
- **Update Rule (vector form):** $\mathbf{x}_{i+1} = \mathbf{x}_i - \eta \nabla f(\mathbf{x}_i)$, where $\mathbf{x} = [x, y]$.
- **Gradient:** $\nabla f(x, y) = [2x - 4, 2y + 2]$.
- **Parameters:** Let's start at $\mathbf{x}_0 = [4, 2]$ with a learning rate $\eta = 0.1$.

Iterations:

- $i = 0$:
 - $\nabla f(\mathbf{x}_0) = [4, 6]$.
 - $\mathbf{x}_1 = \mathbf{x}_0 - \eta \nabla f(\mathbf{x}_0) = [4, 2] - 0.1 \cdot [4, 6] = [3.6, 1.4]$.
- $i = 1$:
 - $\nabla f(\mathbf{x}_1) = [3.2, 4.8]$.
 - $\mathbf{x}_2 = \mathbf{x}_1 - \eta \nabla f(\mathbf{x}_1) = [3.6, 1.4] - 0.1 \cdot [3.2, 4.8] = [3.28, 0.92]$.
- $i = 10$: $\mathbf{x}_{10} = [2.215, -0.678]$

The points are iteratively approaching the minimum at $(2, -1)$.

2D Example: Gradient Descent Visualization



When Analytic Solutions are Tricky

Consider a function involving a sigmoid, common in neural networks:

$$f(x, y) = \text{sigmoid}(4x - 2y) = \frac{1}{1 + e^{-(4x-2y)}}$$

Analytic Approach:

- The gradient is non-linear and more complex:

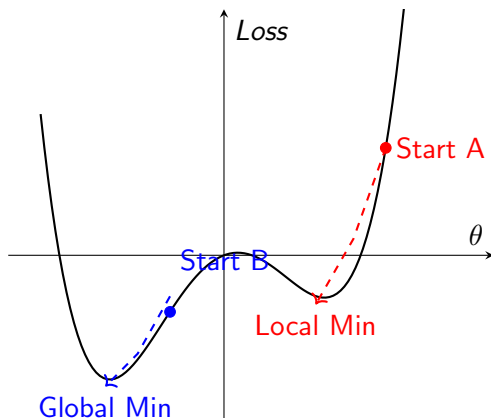
$$\nabla f = \left[\frac{4e^{-(4x-2y)}}{(1 + e^{-(4x-2y)})^2}, \frac{-2e^{-(4x-2y)}}{(1 + e^{-(4x-2y)})^2} \right]$$

- Setting this gradient to $[0, 0]$ does not lead to a simple algebraic solution. The exponential terms make it intractable to solve for x and y directly.

Conclusion

For many real-world functions in AI, especially in deep learning, analytic solutions are impossible or impractical to find. We **must** rely on numerical methods like Gradient Descent.

Real-world loss landscapes are often non-convex, containing multiple valleys (local minima).

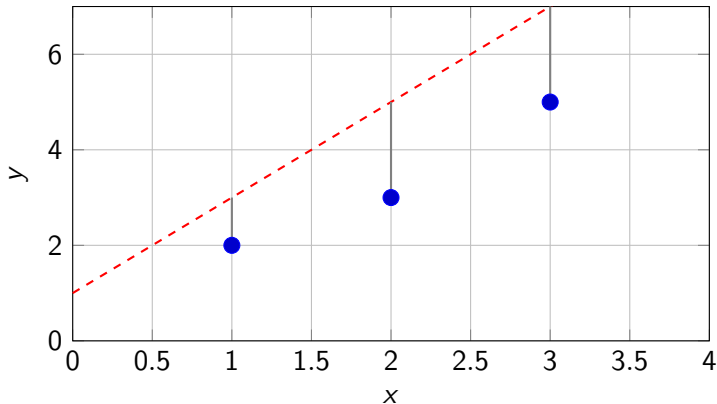


The Initialization Problem:

- **Start A:** Gradient Descent flows into the nearest valley. It gets stuck in a **Local Minimum**.
- **Start B:** Starts in the basin of the **Global Minimum** and converges correctly.
- *Lesson:* Where you start matters!

Problem: Find the best-fit line for the points $(1, 2)$, $(2, 3)$, and $(3, 5)$.

Let's consider an initial guess for the line: $y = 2x + 1$.



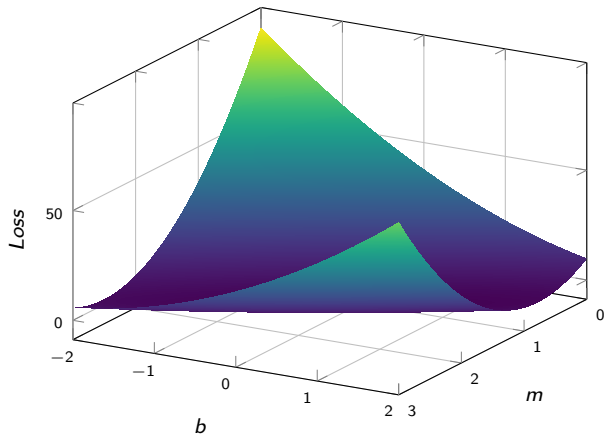
$$\text{Loss} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = 1 + 4 + 4 = 9$$

Deriving the Loss Function Surface

Points: (1, 2), (2, 3), (3, 5)

1. Sum of Squared Errors:

$$\begin{aligned}L(m, b) &= \sum_{i=1}^3 (y_i - \hat{y})^2 \\&= \sum_{i=1}^3 (y_i - mx_i - b)^2 \\&= (2 - m - b)^2 \\&\quad + (3 - 2m - b)^2 \\&\quad + (5 - 3m - b)^2\end{aligned}$$



2. Final Simplification:

$$L = 14m^2 + 12mb + 3b^2 - 46m - 20b + 38$$

The Gradient Vector for the Loss Function

The loss function (SSE):

$$L(m, b) = \sum_{i=1}^3 (y_i - (mx_i + b))^2$$

1. General partial derivatives:

$$\frac{\partial L}{\partial m} = -2 \sum x_i (y_i - mx_i - b)$$
$$\frac{\partial L}{\partial b} = -2 \sum (y_i - mx_i - b)$$

2. For our points (1, 2), (2, 3), (3, 5):

$$\frac{\partial L}{\partial m} = -2 \left(1(2 - m - b) + 2(3 - 2m - b) + 3(5 - 3m - b) \right) = 28m + 12b - 46$$

$$\frac{\partial L}{\partial b} = -2 \left((2 - m - b) + (3 - 2m - b) + (5 - 3m - b) \right) = 12m + 6b - 20$$

So, $\nabla L(m, b) = [28m + 12b - 46, \quad 12m + 6b - 20]$.

We find the minimum by setting the gradient vector to zero, $\nabla L(m, b) = [0, 0]$:

$$28m + 12b - 46 = 0$$

$$12m + 6b - 20 = 0$$

Simplify:

$$14m + 6b = 23$$

$$6m + 3b = 10$$

Solving this 2×2 system gives:

$$m = 1.5, \quad b = \frac{1}{3}$$

The best-fit line is $y = 1.5x + 1/3$.

Solving Numerically with Gradient Descent

For our loss function $L(m, b)$:

- **Update Rule:** $\begin{bmatrix} m_{i+1} \\ b_{i+1} \end{bmatrix} = \begin{bmatrix} m_i \\ b_i \end{bmatrix} - \eta \nabla L(m_i, b_i)$
- **Gradient:** $\nabla L(m, b) = [28m + 12b - 46, \quad 12m + 6b - 20]$
- **Parameters:** Start at $(m_0, b_0) = (2, 1)$ with learning rate $\eta = 0.01$.

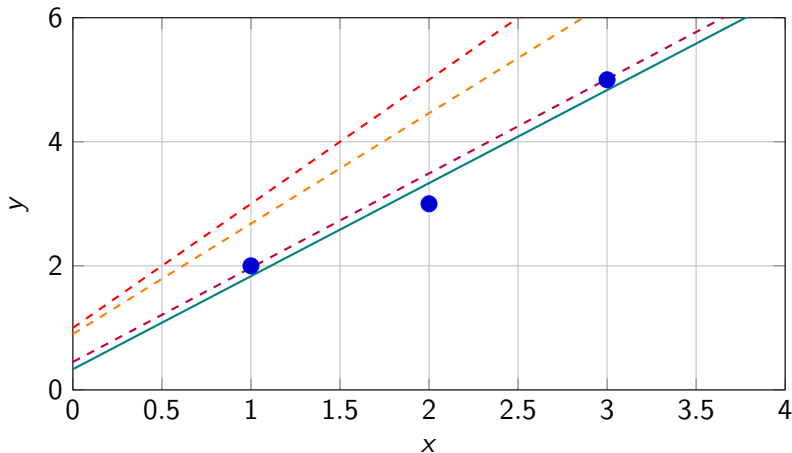
Iterations:

- **i=0:**
 - $\nabla L(2, 1) = [28(2) + 12(1) - 46, \quad 12(2) + 6(1) - 20] = [22, 10]$
 - $\begin{bmatrix} m_1 \\ b_1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - 0.01 \begin{bmatrix} 22 \\ 10 \end{bmatrix} = \begin{bmatrix} 2 - 0.22 \\ 1 - 0.1 \end{bmatrix} = \begin{bmatrix} \mathbf{1.78} \\ \mathbf{0.9} \end{bmatrix}$
- **i=1:**
 - $\nabla L(1.78, 0.9) = [28(1.78) + 12(0.9) - 46, \quad 12(1.78) + 6(0.9) - 20] = [14.64, 6.76]$
 - $\begin{bmatrix} m_2 \\ b_2 \end{bmatrix} = \begin{bmatrix} 1.78 \\ 0.9 \end{bmatrix} - 0.01 \begin{bmatrix} 14.64 \\ 6.76 \end{bmatrix} = \begin{bmatrix} \mathbf{1.6336} \\ \mathbf{0.8324} \end{bmatrix}$
- **i=10:** The parameters continue to approach the analytic solution.

$$[m_{10}, b_{10}] \approx [1.517, 0.452]$$

Visualizing the Solution

The analytic method gives the exact optimal line $y = 1.5x + 1/3$ (solid). Gradient Descent starts with $y = 2x + 1$ (dashed red) and iteratively moves towards the optimal line.



Introduction to Stochastic Gradient Descent (SGD)

Standard Gradient Descent calculates the gradient using **all** data points.

Why is Standard GD sometimes problematic?

- **Speed:** If you have 1,000,000 points, you must sum 1,000,000 errors just to take *one* step.
- **Local Minima:** As seen earlier, you can get stuck in a local valley.

The Solution: SGD

Instead of using all points, use a **Mini-Batch** (e.g., 32 points) or a single point to estimate the gradient.

- **Efficiency:** Updates are much faster.
- **Noise helps:** The gradient is an approximation (noisy). This "jitter" can help kick the parameters out of a local minimum!

The Math Behind SGD: Decomposing the Gradient

The total loss L over the entire dataset of N points is simply the sum of the losses for each individual point i :

$$L(\theta) = \sum_{i=1}^N L_i(\theta)$$

Since the derivative is linear, the total gradient is the sum of individual gradients:

$$\nabla L(\theta) = \sum_{i=1}^N \nabla L_i(\theta)$$

In SGD, instead of summing over all N points, we sum over a small subset (batch) $\mathcal{B}$ of size k :

$$\nabla_{\text{batch}} L(\theta) = \sum_{i \in \mathcal{B}} \nabla L_i(\theta)$$

The SGD Update Formula:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \nabla_{\text{batch}} L(\theta_{\text{old}})$$

Numerical Example: SGD

Task: Linear Regression on noisy data.

Points: (1, 2), (2, 3), (3, 5), (4, 5), (5, 8), (6, 8).

- **Setup:** Model $\hat{y} = mx + b$. Initial $(m, b) = (0.5, 0.5)$. $\eta = 0.02$.
- **Mini-Batch Size:** 2. (Showing first 2 batches).
- **Gradient:** $\nabla_m = -2 \sum x(y - \hat{y})$, $\nabla_b = -2 \sum (y - \hat{y})$.

Batch	Points	Calculation	Update
1	(1, 2), (2, 3)	$(\hat{y} = 0.5x + 0.5): 1, 1.5$ $(y - \hat{y}): 1, 1.5$ $\nabla_m = -2(\textcolor{red}{1}(1) + \textcolor{red}{2}(1.5)) = -8$ $\nabla_b = -2(1 + 1.5) = -5$	$m = 0.5 - 0.02(-8) = \mathbf{0.66}$ $b = 0.5 - 0.02(-5) = \mathbf{0.60}$
2	(3, 5), (4, 5)	$(\hat{y} = 0.66x + 0.6): 2.58, 3.24$ $(y - \hat{y}): 2.42, 1.76$ $\nabla_m = -2(\textcolor{blue}{3}(2.42) + \textcolor{blue}{4}(1.76)) = -28.6$ $\nabla_b = -2(2.42 + 1.76) = -8.36$	$m = 0.66 - 0.02(-28.6) \approx \mathbf{1.23}$ $b = 0.60 - 0.02(-8.36) \approx \mathbf{0.77}$

Visualizing Updates

